

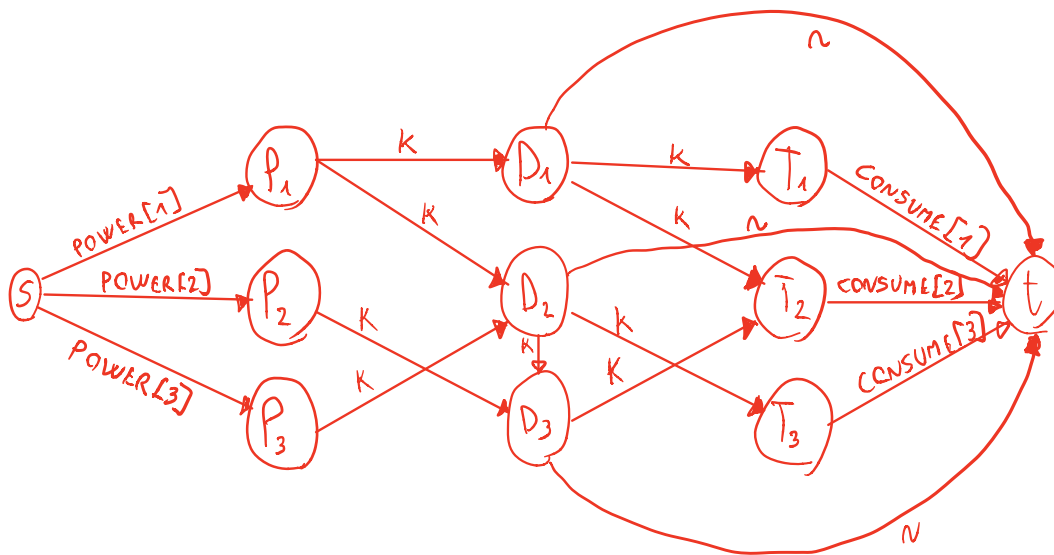
Esercizio B1

Il problema può essere risolto trasformandolo in una rete di flusso.

- Si prende il grafo $G = (V, E)$; per ogni coppia $(u, v) \in V \times V$, si assegna la capacità $c(u, v) = k$ se $(u, v) \in E$, 0 altrimenti.
- Si aggiunge un nodo sorgente s e un nodo superpozzo t .
- Si aggiunge un arco dalla sorgente ad ogni nodo $p \in P$, con peso $power[p]$.
- Si aggiunge un arco da ogni nodo di trasformazione $t \in T$ al superpozzo, con peso $consume[t]$.
- Si aggiunge un arco da ogni nodo di distribuzione $d \in D$ al superpozzo, con peso r .

Si esegue Edmonds-Karp sul grafo, con complessità $O(VE^2)$. Si noti che il limite di Ford-Fulkerson non si applica qui, in quanto le capacità non sono a valori interi.

Se il flusso è pari a $\sum_{t \in T} consume[t] + r \cdot |D|$, è possibile rubare $r \cdot D$ MW senza essere scoperti.



Esercizio B2

L'esercizio può essere risolto tramite backtrack, con costo $O(n \cdot n!)$. Si generano tutte le permutazioni dei valori in S (che sono $n!$) e poi si verifica, una ad una, se sono k -alternate, contandole in maniera ricorsiva. Tale verifica costa $O(n)$, da cui il costo $O(n \cdot n!)$.

```
int kAlternate(SET A, int k)
```

```
    int n = A.size()
```

```
    int[] S = new int[1...n]
```

```
    return kAlternateRec(A, k, S, 1, n)
```

```
int kAlternateRec(SET A, int k, int[] S, int i, int n)
```

```
if i > n then
|   return iif(isKAlternate(S, n, k), 1, 0)
else
|   int tot = 0
|   SET choices = copy(A)
|   foreach v ∈ choices do
|   |   S[i] = v
|   |   A.remove(v)
|   |   tot = tot + kAlternateRec(A, k, S, i + 1, n)
|   |   A.insert(v)
|   return tot
```

La procedura che verifica se una sequenza è k -alternata utilizza due contatori, *countInc* e *countDec*. Il primo elemento può fare parte sia di una sequenza crescente che decrescente, e quindi inizializziamo questa variabile a 1. Dal secondo elemento in poi, quando si incontra un valore crescente, si aumenta il contatore *countInc* e si riporta il contatore *countDec* a 1, per essere pronti a contare gli elementi decrescenti qualora la sequenza crescente si interrompesse. Quando si incontra un valore decrescente, si effettuano le stesse azioni scambiando *countInc* e *countDec*.

```
boolean isKAlternate(int[] S, int n, int k)
```

```
int countInc = 1
int countDec = 1
for i = 2 to n do
|   if S[i - 1] < S[i] then
|   |   countInc = countInc + 1
|   |   if countInc > k then
|   |   |   return false
|   |   countDec = 1
|   countDec = countDec + 1
|   if countDec > k then
|   |   return false
|   countInc = 1
return true
```

Si può ridurre la complessità di un fattore n e ridurre il tempo di calcolo tramite pruning, interrompendo la generazione di una permutazione quando questa non è più k -alternata. Notate che non dovendola verificare al termine, non è necessario salvare le scelte degli elementi nel vettore S . Questa ottimizzazione è stata aggiunta ad una versione precedente grazie al suggerimento di uno studente, Dumitru Damaschin.

```
int kAlternate(SET  $A$ , int  $k$ )
```

```

int  $tot = 0$ 
SET  $choices = A$ 
foreach  $v \in choices$  do
     $A.remove(v)$ 
     $tot = tot + kAlternateRec(A, k, v, A.size() - 1, 1, 1)$ 
     $A.insert(v)$ 
return  $tot$ 
```

```
int kAlternateRec(SET  $A$ , int  $k$ , int  $prev$ , int  $i$ , int  $countInc$ , int  $countDec$ )
```

```

if  $i == 0$  then
    return 1
int  $tot = 0$ 
SET  $choices = copy(A)$ 
foreach  $v \in choices$  do
     $A.remove(v)$ 
    if  $prev < v$  and  $countInc < k$  then
         $tot = tot + kAlternateRec(A, k, v, i - 1, countInc + 1, 1)$ 
    if  $prev > v$  and  $countDec < k$  then
         $tot = tot + kAlternateRec(A, k, v, i - 1, 1, countDec + 1)$ 
     $A.insert(v)$ 
return  $tot$ 
```

Si noti che il primo elemento viene scelto liberamente fra gli n elementi disponibili, e per questo viene scelto fuori dalla procedura ricorsiva. Dal secondo elemento in poi, un valore viene aggiunto solo se non crea sequenze lunghe $k + 1$ o più.

Esercizio B3

Il problema può essere risolto con programmazione dinamica. Sia $DP[i]$ il valore della sottosequenza crescente massimale contenuta nei primi i elementi di A e che termina nella posizione i -esima. $DP[i]$ può essere calcolata nel modo seguente:

$$DP[i] = \begin{cases} A[i] & \forall 1 \leq j < i : A[j] < A[i] \\ \max_{1 \leq j < i : A[j] < A[i]} \{DP[j] + A[i]\} & \text{altrimenti} \end{cases}$$

Questa equazione ricorsiva può essere tradotta in codice nel modo seguente, tramite programmazione dinamica. Il valore cercato è il massimo fra tutti gli elementi della tabella DP , in quanto la sottosequenza cercata può terminare in uno qualunque delle posizioni.

```

int maxIncreasing(int[] A, int n)


---


  int[] DP = new int[1...n]
  for i = 1 to n do
    DP[i] = A[i]
    for j = 1 to i - 1 do
      if A[j] < A[i] then
        DP[i] = max(DP[i], DP[j] + A[i])
  return max(DP, n)

```

La complessità dell'algoritmo proposto è $\Theta(n^2)$.